

Roteiro de Apresentação — Vitrine Largo da Ordem

Roteiro da Apresentação — Vitrine Virtual da Feira do Largo da Ordem

Disciplina: Mobile Development iOS — PUCPR 2026 **Avaliação:** Somativa SwiftUI (foco em Acessibilidade) **Duração alvo:** 14–16 minutos (limite permitido: 10–20 min) **Integrantes:** Lucas Stopinski da Silva • Lucas Bruno e Silva • Pedro Henrique Silva Guligurski

Como usar este roteiro

- Cada bloco indica **quem fala, tempo aproximado, o que mostrar na tela e pontos a cobrir** (fale natural, não leia o roteiro literalmente).
 - Pré-requisitos da gravação:
 - Xcode aberto com o projeto carregado
 - Simulador **iPhone 17 (iOS 26+)** rodando o app
 - Simulador **iPad Pro 13"** disponível para mostrar responsividade (bloco 8)
 - **Configurações de Acessibilidade prontas** no simulador (VoiceOver e Dynamic Type) — instruções no checklist abaixo
 - Como o foco da nota é A11y (30% do peso), reserve **3-4 minutos** para a demonstração de acessibilidade no simulador.
-

Estrutura geral

Bloco	Tempo	Quem fala	Tema
1	0:00 – 0:30	Lucas Stopinski	Abertura e apresentação
2	0:30 – 1:30	Pedro Henrique	Contexto, tema e objetivos
3	1:30 – 4:00	Pedro Henrique	Demo visual no simulador
4	4:00 – 7:30	Lucas Bruno	Demo de Acessibilidade (VoiceOver + Dynamic Type)
5	7:30 – 8:30	Lucas Stopinski	Arquitetura MVVM e estrutura de pastas
6	8:30 – 9:30	Lucas Bruno	Model <code>ProdutoArtesanal</code> + propriedade <code>precoAcessivel</code>
7	9:30 – 11:00	Pedro Henrique	<code>LazyVGrid</code> adaptativo + <code>.searchable</code>
8	11:00 – 12:30	Lucas Bruno	<code>BotaoFavoritoView</code> (44×44) + <code>ProdutoCardView</code>
9	12:30 – 13:30	Lucas Stopinski	<code>DetalhesProdutoView</code> + <code>accessibilitySortPriority</code>
10	13:30 – 14:30	Pedro Henrique	Dificuldades encontradas
11	14:30 – 15:00	Lucas Stopinski	Encerramento

Bloco 1 — Abertura (0:00 – 0:30) — Lucas Stopinski

Mostrar: webcam ou slide simples com nome do projeto.

Falar:

- "Olá, professor. Somos o Lucas Stopinski, o Lucas Bruno e o Pedro Henrique, da disciplina de Mobile Development iOS da PUCPR."
- "Este é nosso projeto somativo de SwiftUI: a **Vitrine Virtual da Feira do Largo da Ordem**."
- "Nos próximos 15 minutos vamos demonstrar o app, com foco especial em **acessibilidade** — que vale 30% da nota — e fazer um walkthrough da arquitetura e do código."

Bloco 2 — Contexto e objetivos (0:30 – 1:30) — Pedro Henrique

Mostrar: README aberto na seção de objetivos, ou um slide com o tema.

Falar:

- "O objetivo era construir um catálogo de e-commerce em SwiftUI com tema regional curitibano — a Feira do Largo da Ordem."
- "Trouxemos 12 produtos típicos divididos em 7 categorias: artesanato em madeira, comidas, arte, vestuário, antiguidades, acessórios e beleza."
- "Os requisitos obrigatórios envolvem `LazyVGrid` adaptativo, `NavigationLink` para tela de detalhes, `.searchable` para busca, gerenciamento de estado dos favoritos e — o mais importante — implementação **rigorosa** das diretrizes de acessibilidade da Apple."
- "Vamos mostrar primeiro o app rodando, depois a parte de A11y, e por fim o código."

Bloco 3 — Demo visual (1:30 – 4:00) — Pedro Henrique

Mostrar: simulador iPhone 17 com o app aberto.

Roteiro da demo:

1. Tela principal (~30s)

- "Esta é a vitrine. Note que o `LazyVGrid` mostra 2 colunas no iPhone retrato, com cards padronizados."
- Faça scroll suave: "12 produtos divididos em categorias, cada card com imagem em SF Symbol, nome, categoria, preço e o botão de favoritar."

2. Favoritar (~30s)

- Toque no coração de um produto: "O ícone muda imediatamente com uma animação `symbolEffect(.bounce)`."
- Toque em outros 2-3: "O estado é gerenciado por um `@StateObject` no ViewModel — totalmente reativo."
- Toque de novo no primeiro: "Alterna de volta, sem reload."

3. Busca (~30s)

- Toque na barra de busca, digite "mate": "Filtra por nome — aparecem só os produtos com 'mate' no nome."
- Apague e digite "madeira": "Filtra por categoria também — aparecem as peças da categoria Madeira."
- Apague: "Tudo volta. Quando o resultado é vazio, mostramos um `ContentUnavailableView` nativo."
- Demonstre uma busca sem resultados (ex: "xyz"): "Estado vazio com mensagem útil."

4. Tela de detalhes (~50s)

- Toque em um card: "A navegação acontece em `NavigationLink` — note que tocar no card abre os detalhes, mas tocar no coração NÃO abre, apenas alterna o favorito."
- "Aqui temos o nome em destaque, imagem ampliada com gradiente, metadados em card (artesanato, categoria, preço), descrição completa e o botão de contato."
- Toque em "Entrar em contato com o Artesão": "Botão fictício que mostra confirmação de contato."
- Volte para a vitrine.

Bloco 4 — Demo de Acessibilidade (4:00 – 7:30) — Lucas Bruno

⚠ **Este é o bloco mais importante — 30% da nota.** Dê tempo, mostre cada item com calma.

Mostrar: simulador iPhone com VoiceOver e depois Dynamic Type.

Parte 1 — VoiceOver (4:00 – 6:00)

Preparação: ative o VoiceOver no simulador via **Hardware** → **Accessibility Shortcut**, ou no terminal: `xcrun simctl spawn booted launchctl setenv UIAccessibilityVoiceOverEnabled YES` (não funciona em todos os SO; alternativa é ir em Settings → Accessibility → VoiceOver).

Falar e demonstrar:

- "Vou ativar o VoiceOver agora — ele lê em voz alta o que está em foco na tela."
- **Foque no primeiro card:** "Ouçam o que ele lê: 'Imagem ilustrativa de Escultura de Capivara, produzida por Sebastião Andrade'. A imagem tem um `accessibilityLabel` descritivo, não genérico."
- Avance para o preço: "Aqui está o detalhe que destacamos: o preço é lido como '**Preço: 85 reais**', não 'R cifrão 85 ponto 00' que seria o comportamento padrão."
- Mostre o código rapidamente: `.accessibilityLabel(produto.precoAcessivel)` que vem de uma propriedade no model.
- Avance para o botão: "O botão de favoritar é lido como 'Adicionar Escultura de Capivara aos favoritos. Botão.' — note que o label é dinâmico: muda para 'Remover dos favoritos' quando já está favoritado."
- **Dê duplo-toque no botão:** "Toque duplo no VoiceOver alterna o favorito. Veja como o label foi atualizado."
- **Abra a tela de detalhes com swipe + double-tap:** "Aqui aplicamos `accessibilitySortPriority` — o VoiceOver lê o **Nome do produto primeiro**, depois os metadados, descrição, imagem e por último o botão de contato. Mesmo que visualmente a imagem esteja no topo, a leitura segue uma ordem lógica."

Parte 2 — Dynamic Type (6:00 – 7:30)

Preparação: Settings → Accessibility → Display & Text Size → Larger Text → ativar e arrastar para o maior tamanho.

- Desative o VoiceOver primeiro.
- Volte para a vitrine.
- "Agora vou aumentar o tamanho da fonte do sistema para o máximo acessível."
- Mostre a vitrine com fonte gigante: "Note que todos os textos crescem — nome, categoria, preço — e o layout se adapta. Nenhum texto está com `.frame(height:)` fixo."
- "Mas tem mais: as imagens dos cards também crescem proporcionalmente, graças ao `@ScaledMetric`."
- Mostre o código: `@ScaledMetric(relativeTo: .body) private var alturaImagem: CGFloat = 130`.
- Toque em um card pra abrir os detalhes com fonte grande: "Os textos longos quebram em várias linhas sem cortar, porque usamos `multilineTextAlignment` e `fixedSize(horizontal: false, vertical: true)`."
- Volte ao tamanho normal: "Pronto, isso é Dynamic Type funcionando de verdade."

Bloco 5 — Arquitetura MVVM (7:30 – 8:30) — Lucas Stopinski

Mostrar: Project Navigator do Xcode com pastas expandidas.

Falar:

- "Decidimos por **MVVM** — Model-View-ViewModel — porque é o padrão idiomático do SwiftUI."
- Aponte para cada pasta:
 - `Models/` → "Define o `ProdutoArtesanal` puro, sem nenhuma dependência de SwiftUI."
 - `Data/` → "Camada de dados estática — array de produtos mock. Fácil de substituir por uma API futura."
 - `ViewModels/` → "O `VitrineViewModel` é um `ObservableObject` que gerencia a lista, a busca e os favoritos."
 - `Views/` → "Aqui está toda a parte visual. Note que extraímos `ProdutoCardView`, `BotaoFavoritoView` e `DetalhesProdutoView` como Views separadas — não está tudo em um arquivo só."
- "O fluxo é: a `VitrineView` instancia o ViewModel como `@StateObject`, observa as mudanças e propaga ações via closures."

Bloco 6 — Model e preço acessível (8:30 – 9:30) — Lucas Bruno

Mostrar: `Models/ProdutoArtesanal.swift`.

Falar:

- "O struct tem exatamente os 8 campos que o enunciado pedia, com `id: UUID` conformando a `Identifiable`."
- "O `isFavorito` é `var` (não `let`) para permitir mutação, com valor padrão `false` no init."
- Aponte para `precoFormatado`: "Aqui temos a formatação visual com `NumberFormatter` locale `pt_BR` — produz 'R\$ 85,00'."
- Aponte para `precoAcessivel`: "E aqui está o pulo do gato — uma propriedade separada que produz texto natural para o VoiceOver: 'Preço: 85 reais' ou 'Preço: 22 reais e 50 centavos' quando tem centavos."
- "Manter essa lógica no model, e não na view, é uma boa prática — assim qualquer view que precisar da string acessível tem acesso pronta."

Bloco 7 — LazyVGrid + Searchable (9:30 – 11:00) — Pedro Henrique

Mostrar: `Views/VitrineView.swift`.

Falar:

1. Setup do grid (~30s)

- Aponte para `colunas = [GridItem(.adaptive(minimum: 150), spacing: 16)]`: "Esta é a chave da responsividade. O `.adaptive(minimum: 150)` diz ao SwiftUI para criar quantas colunas couberem com pelo menos 150 pontos cada."
- "Em iPhone retrato: 2 colunas. Landscape: 3. iPad: 3 ou 4. Sem precisar checar device."

2. NavigationStack + ScrollView + LazyVGrid (~30s)

- "O `ScrollView` envolve o `LazyVGrid` porque grade sozinha não rola. O 'Lazy' significa que as células fora da viewport não são instanciadas — performance ótima mesmo com centenas de produtos."
- Mostre o `ForEach`: "Iteramos sobre `produtosFiltrados`, que é uma computed property do ViewModel."

3. Searchable (~30s)

- Aponte para `.searchable(text: $viewModel.termoBusca, ...)`: "Modificador nativo do SwiftUI 4+. O binding `$viewModel.termoBusca` é two-way, e a filtragem acontece automaticamente quando o ViewModel atualiza `produtosFiltrados`."
- Mostre o método no ViewModel: "Filtramos por `nome` ou `categoria` com `localizedCaseInsensitiveContains` — acentos e maiúsculas são ignorados."

Bloco 8 — Botão Favorito e Card (11:00 – 12:30) — Lucas Bruno

Mostrar: `Views/BotaoFavoritoView.swift` e depois `Views/ProdutoCardView.swift`.

Falar sobre BotaoFavoritoView:

- "Este componente foi extraído justamente para garantir os requisitos de acessibilidade em um único lugar."
- Aponte para `.frame(minWidth: 44, minHeight: 44)`: "Aqui está o touch target de 44 pontos exigido pelo enunciado."
- `.contentShape(Rectangle())`: "Sem isso, o toque só seria capturado no pixel do ícone, mesmo com o frame maior. Esse modificador define a área de hit-test."
- `.buttonStyle(.plain)`: "Necessário porque este botão fica dentro de um `NavigationLink`. Sem isso, o link engole o tap."
- `.accessibilityLabel(...)`: "Label dinâmico — muda entre 'Adicionar X aos favoritos' e 'Remover X dos favoritos'."
- `.accessibilityHint("Toque duplo para alternar.")`: "Hint contextual conforme recomendação da Apple."

Falar sobre ProdutoCardView:

- "Aqui está a montagem do card. Note que toda a tipografia usa fontes semânticas: `.subheadline`, `.caption`, `.headline` — nada de tamanhos fixos."
- "A altura da imagem usa `@ScaledMetric` para escalar com Dynamic Type."
- "Cada label de acessibilidade é cuidadoso — a categoria, por exemplo, é lida como 'Categoria: Madeira' e não só 'Madeira', dando contexto."

Mostrar VitrineView de novo brevemente:

- "E aqui, no `ForEach`, vê-se como o card é envelopado em um `NavigationLink` com `.buttonStyle(.plain)` — sem isso, o card ficaria com o destaque azul padrão de link."

Bloco 9 — Detalhes e Sort Priority (12:30 – 13:30) — Lucas Stopinski

Mostrar: `Views/DetalhesProdutoView.swift`.

Falar:

- "A tela de detalhes é um `ScrollView` com `VStack`, contendo: nome, imagem, metadados em card, descrição em card, e botão de contato."
- **Foco em** `accessibilitySortPriority` (~40s):

- "Visualmente o nome aparece primeiro, depois a imagem. Mas o `accessibilitySortPriority` vai além disso — ele controla a **ordem de leitura** do VoiceOver."
 - Mostre: Nome com `(10)`, Metadados `(8)`, Descrição `(6)`, Imagem `(5)`, Botão `(4)`.
 - "Prioridade maior é lida primeiro. Então mesmo se a gente mudar a ordem visual, a leitura linear continua: nome → metadados → descrição → imagem → botão. Conforme exige o enunciado."
 - Mostre `metadados`: "Note o `accessibilityElement(children: .combine)` em cada linha — isso faz o VoiceOver ler 'Artesão: Sebastião Andrade' como uma frase única, em vez de 'Artesão' (pausa) 'Sebastião Andrade'."
 - Mostre o botão de contato: "É um `Button` com `Label` (ícone + texto), preenche a largura toda com `frame(maxWidth: .infinity)`, altura mínima de 50 — confortável tocar."
-

Bloco 10 — Dificuldades (13:30 – 14:30) — Pedro Henrique

Mostrar: README aberto na seção "Dificuldades encontradas" ou continuar no código.

Falar — destaque 3 pontos rápidos:

1. "**Botão dentro de NavigationLink** — o tap do link engolia o botão. Resolvido com `.buttonStyle(.plain)` em ambos."
 2. "**Leitura literal do preço** — `R$ 85,00` virava 'R cifrão' no VoiceOver. Criamos `precoAcessivel` no model retornando 'Preço: 85 reais' e aplicamos como `accessibilityLabel`."
 3. "**Touch target real** — só `frame(minWidth: 44)` não basta. Precisou de `.contentShape(Rectangle())` pra estender a área de hit-test."
-

Bloco 11 — Encerramento (14:30 – 15:00) — Lucas Stopinski

Mostrar: simulador com a vitrine aberta, ou um slide final.

Falar:

- "Resumindo: o app cumpre todos os requisitos funcionais, com arquitetura MVVM limpa, totalmente em SwiftUI, e — o que mais nos orgulha — implementação rigorosa das diretrizes de acessibilidade da Apple."
 - "O código está no GitHub e o relatório técnico em PDF acompanha esta entrega."
 - "Agradecemos pela atenção, professor!"
-

Checklist antes de gravar

Ambiente

- ☐ Xcode 26+ aberto com `VitrineLargoOrdem.xcodeproj`
- ☐ Simulador **iPhone 17 (iOS 26.4)** com o app instalado e rodando
- ☐ Simulador **iPad Pro 13"** disponível como segunda opção
- ☐ Resolução de gravação: 1080p ou superior

Acessibilidade preparada no simulador

- ☐ **Atalho do VoiceOver:** Settings → Accessibility → Accessibility Shortcut → marcar **VoiceOver**. Isso permite ativar/desativar com **3 cliques no Lateral/Home**.
- ☐ **Dynamic Type:** Settings → Accessibility → Display & Text Size → Larger Text. Aprenda a navegar até o slider antes da gravação.
- ☐ **Comandos do VoiceOver no simulador:**
 - Selecionar próximo: **Ctrl + Opt + ⌘ + →** (com a interação do simulador habilitada via Hardware → Keyboard → Connect Hardware Keyboard)
 - Ativar item selecionado: **Ctrl + Opt + Space**

Ensaio

- ☐ Fazer um ensaio completo cronometrado para garantir que cabe em 15 minutos
- ☐ Verificar áudio (sem eco, sem ruído de fundo)
- ☐ Combinar transições entre apresentadores ("passo pra você, Bruno..." / "te devolvo, Pedro...")

Dicas finais

- **Divisão de tela:** Xcode/simulador ocupando ~70%, webcam dos apresentadores em PiP no canto inferior direito.
- **Se errar uma fala:** pause 2 segundos e refaça. Na edição, corta o trecho ruim.
- **Atalhos úteis do Xcode:**
 - `Cmd + 0`: esconder Navigator (mais espaço para o código)
 - `Cmd + Opt + 0`: esconder Inspector
 - `Cmd + Shift + 0`: abrir arquivo rapidamente
- **Subir no YouTube como "Não Listado"** — qualquer um com o link acessa, mas não aparece em buscas.